CITY UNIVERSITY OF SCIENCE & INFORMATION TECHNOLOGY

Department of Computer Science

ASSIGNMENT #1

Software Engineering: Processes and Development

Name	Arsalan Zafar
Student ID	_____
Section	_____
Course	Software Engineering: Processes and Development
Submission Date	_____

Question 1: Software Construction and Software Development

a. Define software construction and software development

Software construction is the part of software engineering concerned with creating the actual software product. It includes writing source code, debugging, testing individual parts, integrating components, and managing software versions.

Software development is the broader process of producing and maintaining software. It includes activities such as requirements gathering, analysis, design, construction, testing, deployment, and maintenance.

b. Major activities involved in software construction

- **Coding:** Translating requirements and designs into executable source code.
- **Debugging:** Finding and correcting defects in program behavior.
- **Testing:** Checking software components to detect defects and verify expected behavior.
- **Integration:** Combining separate components and checking that they work together.
- **Configuration management:** Controlling versions and changes to source code, dependencies, documents, and releases.
- **Build and release:** Compiling, packaging, and preparing software for testing or deployment.

c. Difference between software construction and software development

Aspect	Construction	Development
Scope	Building the software product	Complete software life cycle
Main work	Coding, debugging, testing, integration, configuration	Requirements, analysis, design, construction, testing, deployment, maintenan
Relationship	A major part of development	Broader process containing construction
Example	Implementing a login module	Developing a complete university portal

d. Role of major construction activities

- **Coding** creates the executable implementation.
- **Debugging** removes defects and incorrect behavior.
- **Testing** provides evidence that the software meets requirements and helps find defects.
- **Integration** ensures modules and services work correctly together.
- **Configuration management** tracks and controls changes, versions, and releases.

e. Real-world example

Consider a **university student management system**. Developers implement login, registration, course enrollment, fees, and results. They debug incorrect calculations and database queries, test individual modules, integrate the modules with the database and user interface, and use a version-control system such as Git to manage changes. These activities continue as

new features and fixes are introduced.

Question 2: Software Engineering Process Infrastructure

a. Definition of SEPI

Software Engineering Process Infrastructure (SEPI) is the organizational environment that supports the definition, execution, monitoring, and improvement of software engineering processes. It includes standards, procedures, guidelines, tools, documentation, roles, training, and management support.

b. Major components

- **Standards:** Common rules or accepted practices for coding, testing, documentation, and quality.
- **Guidelines:** Recommended practices for consistent decisions.
- **Processes and procedures:** Defined steps for reviews, testing, releases, and changes.
- **Tools:** Version control, issue tracking, testing, build, and project-management tools.
- **Documentation:** Process descriptions, templates, checklists, requirements, and test records.
- **Roles and responsibilities:** Clear ownership for developers, testers, managers, and reviewers.
- **Training and organizational support:** Resources and management commitment.
- **Measurement and improvement:** Metrics, audits, feedback, and lessons learned.

c. Importance of standards, guidelines, procedures, tools, documentation, and support

Standards create consistency. Guidelines help people apply standards in practical situations. Procedures explain how recurring activities should be performed. Tools automate or support development work. Documentation preserves knowledge and provides evidence of decisions and results. Organizational support provides time, training, resources, and management commitment.

d. How SEPI improves quality and productivity

A strong process infrastructure helps teams detect defects earlier, control changes, reuse proven practices, and work consistently. For example, code review can find problems before release, while automated tests reduce repeated manual checking. Clear documentation also makes it easier for new team members to understand the project.

e. Diagram of SEPI

SOFTWARE ENGINEERING PROCESS INFRASTRUCTURE		
Standards & Guidelines	Processes & Procedures	Tools & Automation
Documentation & Templates	Roles, Training & Support	Measurement & Improvement

Question 3: Systems Engineering Life Cycle Models

a. Definition

A **systems engineering life cycle model** is a structured way of organizing the stages used to develop, verify, deliver, operate, and maintain a system. Models differ in how they handle sequence, iteration, risk, change, and customer feedback.

b. Life cycle models

Waterfall Model

Waterfall follows a mostly sequential approach: requirements, design, implementation, testing, deployment, and maintenance. It is appropriate when requirements are stable and well understood.

Requirements
↓

Design
↓
Implementation
↓
Testing
↓
Deployment & Maintenance

V-Model

The V-Model emphasizes verification and validation. Development stages are paired with corresponding testing activities. Testing is planned early rather than being treated only as a final activity.

Development	Testing / Validation
Requirements	Acceptance / System Testing
System Design	System Testing
Architecture	Integration Testing
Detailed Design	Unit Testing
Implementation	Test Execution

Incremental Model

The Incremental model develops a system in several usable releases. Each increment adds functionality, allowing useful features to be delivered earlier and feedback to influence later increments.

Increment 1: Login	→	Increment 2: Registration	→	Increment 3: Results & Reports

Spiral Model

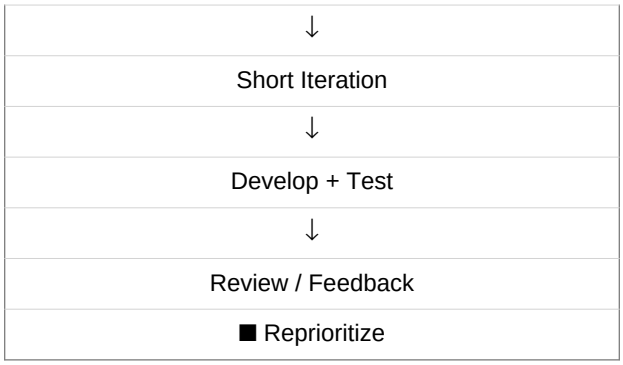
The Spiral model combines iterative development with explicit risk analysis. Each cycle includes planning, risk analysis, development, and evaluation. It is useful for large, complex, or high-risk projects.

1. Planning
↓
2. Risk Analysis
↓
3. Development & Verification
↓
4. Customer Evaluation
■ Next Cycle

Agile Model

Agile uses short iterations, frequent testing, and regular customer or stakeholder feedback. Requirements can evolve as the team learns more. It is commonly used for web, mobile, and SaaS products with changing priorities.

Backlog / Priorities



c. Comparison

Model	Approach	Flexibility	Risk	Customer involvement	Typical suitability
Waterfall	Sequential	Low	Planned early	Lower after requirements	Stable requirements
V-Model	Sequential + testing	Low	Verification planned early	Moderate	Quality/verification-focused systems
Incremental	Repeated releases	Medium–High	Handled per increment	Regular	Systems deliverable in modules
Spiral	Iterative + risk-driven	High	Explicit risk analysis	Frequent	Large/high-risk systems
Agile	Iterative + incremental	High	Short cycles + feedback	High	Changing requirements

d. Suitable project types and examples

- **Waterfall:** Stable, well-documented, contract-driven projects.
- **V-Model:** Systems requiring strong verification and validation, including highly regulated or safety-critical contexts.
- **Incremental:** Applications that can deliver separate modules, such as a university portal or e-commerce system.
- **Spiral:** Large or technically uncertain projects where risk analysis is essential.
- **Agile:** Web, mobile, and SaaS products where customer needs and priorities can change.

e. Diagram explanation

Waterfall moves through major stages in sequence. The V-Model links development activities with corresponding testing activities. Incremental development grows the product through usable releases. Spiral repeatedly examines risk while developing the system. Agile repeatedly develops and reviews small pieces of functionality using feedback.

Question 4: Process Implementation and Levels of Process Definition

a. Definition of process implementation

Process implementation means putting a defined software engineering process into actual use within an organization or project. It requires appropriate people, tools, procedures, training, measurements, and management support.

b. Steps for implementing a software engineering process

1. Assess the current process and identify improvement needs.

2. Define the target process, roles, activities, inputs, outputs, and quality criteria.

3. Prepare procedures, templates, checklists, and documentation.

4. Select and configure supporting tools.

5. Train team members and communicate responsibilities.

6. Pilot the process on a suitable project.

7. Monitor results using reviews and measurements.

8. Collect feedback and improve the process.

c. Process definition and its importance

Process definition is the documented description of how software engineering work should be performed. It identifies activities, roles, inputs, outputs, standards, tools, and controls. It creates a common understanding, improves consistency, supports training and reviews, and provides a basis for measurement and improvement.

d. Levels of process definition

Level	Meaning
Organizational / Standard Process	Common process established by the organization for use across projects.
Defined Process	Detailed description of how a process is organized and applied in a particular context.
Project-Specific Process	Tailored process selected for one project based on size, risk, technology, customer needs, and constraints.
Individual / Task-Level Process	Detailed instructions or practices used for a specific task by an individual or small team.

e. Tailoring an organizational process

A standard process should not be applied blindly to every project. The project team examines the organizational process, identifies mandatory and optional activities, and adjusts the process to fit project size, risk, technology, customer needs, and constraints. Tailoring decisions should be documented while required quality and organizational controls are retained.

f. Practical example

Suppose a software company requires requirements review, design review, code review, unit testing, integration testing, release approval, and configuration management. For a small three-month university mobile-app project, the team can keep these essential controls but use a lightweight requirements template, combine some meetings, and conduct reviews online. Version control, test results, and release decisions are still recorded. This shows how a standard organizational process can be tailored without removing important controls.

Overall Conclusion

Software engineering processes provide a systematic way to develop reliable software. Software construction focuses on building the product, while software development covers the broader life cycle. Process infrastructure provides standards, tools, documentation, and organizational support. Life cycle models organize development differently according to project needs, while process implementation and tailoring allow organizations to apply defined practices in a practical and controlled way.

References

- Ian Sommerville, *Software Engineering*, Pearson.
- Roger S. Pressman and Bruce R. Maxim, *Software Engineering: A Practitioner's Approach*, McGraw-Hill.
- Pankaj Jalote, *An Integrated Approach to Software Engineering*, Springer.
- ISO/IEC/IEEE 12207, *Systems and software engineering — Software life cycle processes*.
- Agile Manifesto, https://agilemanifesto.org/